RPG ARENA

PLANO MESTRE DE 6 MESES

Backend open source em Java 21 e Spring Boot

24 semanas | 144 tarefas | 12 episódios | 24 Shorts

24/08/2026 a 07/02/2027
Carga: 18-22 horas por semana
Meta: release v1.0.0 pública e documentada

Como usar este plano

1. Execute uma semana por vez; não pule o critério de aceite.
2. Atualize o Excel somente pela coluna Status.
3. Use um PR por tarefa ou grupo pequeno e anexe a evidência.
4. Grave durante a implementação; não tente reconstruir a história depois.
5. Publique um episódio a cada duas semanas e um Short por semana.
6. Corte escopo opcional antes de sacrificar segurança, transação ou testes.

PERÍODO	CARGA	BACKLOG	CONTEÚDO
24 semanas	18-22h/semana	144 tarefas	12 episódios + 24 Shorts

Conteúdo do pacote

- Escopo do produto, funcionalidades, regras, endpoints e modelo de dados.
- Arquitetura modular, estrutura de pastas e decisões obrigatórias.
- Roadmap mensal e 24 semanas com tarefas, aceite, evidência e fontes.
- Minissérie completa, episódios, Shorts, roteiro e checklist de gravação.
- Método de execução, Git flow, Definition of Ready e Definition of Done.
- Excel com progresso automático e arquivos .tarefas semanais.
- Materiais prontos para README, contribuição, segurança e templates GitHub.

Escopo do projeto

Open-source turn-based RPG backend built with Java 21 and Spring Boot to practice domain modeling, REST APIs, PostgreSQL, security, testing, Docker, and software architecture.

Incluído na versão 1.0

- Contas, JWT, refresh, roles e ownership.
- Personagem, classes, atributos, estados, XP e habilidades.
- Combate por turnos, itens, efeitos e histórico.
- Inventário, equipamento, loot e transações.
- Missões, objetivos, recompensas e conteúdo versionado.
- Mundo, viagens, encontros, NPCs, comerciantes e economia.
- Administração, OpenAPI, Bruno, Docker, CI, métricas e deploy.

Fora do escopo

Interface gráfica completa, multiplayer, PvP, Kafka, microserviços, Kubernetes, pagamentos e IA generativa dentro do jogo.

Arquitetura

Monólito modular. Cada domínio contém domain, application, infrastructure e interfaces. O domínio não depende de Spring, JPA ou HTTP. Application controla casos de uso e transações; infrastructure implementa portas; interfaces traduz contratos externos.

Módulos

account, character, combat, inventory, progression, quest, world, economy e shared.

Roadmap mensal

Mês	Fase	Objetivo	Marco	Prova
1	Núcleo jogável	Fechar escopo, arquitetura, personagem e combate determinístico.	v0.1.0 - batalha jogável sem banco	Testes demonstram criação de personagem e batalha completa.
2	Persistência e progressão	Persistir dados e entregar inventário, equipamentos, XP, habilidades e missões.	v0.2.0 - jornada persistente	Banco nasce por migrations e a jornada sobrevive a reinício.
3	API e segurança	Expor o jogo por REST com autenticação, autorização e ownership.	v0.3.0 - API segura	Bruno executa cadastro, login, personagem, batalha e missão.
4	Mundo e conteúdo	Adicionar exploração, encontros, NPCs, economia e missões ramificadas.	v0.4.0 - sistemas de mundo	Fluxo ponta a ponta inclui viagem, compra e escolha narrativa.
5	Qualidade e operação	Elevar testes, Docker, documentação, observabilidade, desempenho e segurança.	v1.0.0-rc.1 - candidato a release	CI, Docker, OpenAPI, métricas e quality gates funcionam.
6	Publicação e portfólio	Publicar, estabilizar, abrir colaboração e transformar o projeto em prova profissional.	v1.0.0 - release pública	Projeto online, documentado, demonstrável e usado em candidaturas.

Mês 1 - Núcleo jogável

Objetivo: Fechar escopo, arquitetura, personagem e combate determinístico.

Marco: v0.1.0 - batalha jogável sem banco

Prova: Testes demonstram criação de personagem e batalha completa.

Semana 01 - Produto, escopo e repositório

Objetivo: Transformar a ideia em um produto executável e público.

Entrega: Repositório preparado, backlog priorizado e escopo fechado.

RPG-M1-W01-01	Definir visão e público Escreva a proposta, público, fantasia central, diferencial educacional e resultado esperado em uma página. GitHub Documentation	Aceite: A visão responde para quem, por quê, o que será entregue e como medir sucesso. Evidência: docs/product/vision.md	3
RPG-M1-W01-02	Fechar MVP e não objetivos Inclua conta, personagem, combate, inventário, progressão, missão, mundo e economia; exclua UI gráfica, multiplayer, Kafka e microserviços. GitHub Documentation	Aceite: Todo item do MVP cabe em um dos seis marcos e o fora de escopo está explícito. Evidência: docs/product/scope.md	3
RPG-M1-W01-03	Descrever jornadas e glossário Modele criar conta, criar personagem, lutar, obter item, evoluir, viajar, comprar e concluir missão; defina termos do jogo. Mermaid Documentation	Aceite: Cada jornada tem fluxo principal, falhas, pré-condições e resultado. Evidência: docs/requirements/journeys.md e glossary.md	4
RPG-M1-W01-04	Criar repositório e metadados Use nome rpg-arena, descrição, topics, licença MIT, .gitignore Java, main protegida e squash merge. GitHub Documentation	Aceite: Repositório público apresenta descrição, licença, topics e regras de merge. Evidência: Repositório GitHub configurado	3
RPG-M1-W01-05	Criar backlog, DoR e DoD Transforme o escopo em issues; adote Ready, In Progress, Review e Done; registre critérios para iniciar e concluir. GitHub Documentation	Aceite: Todas as issues do mês 1 têm aceite, prioridade, tamanho e evidência. Evidência: GitHub Project e docs/process.md	4
RPG-M1-W01-06	Preparar identidade e trailer da série Defina nome da série, padrão de thumbnail, tela de código, vinheta curta e grave um trailer de 45-60 segundos. YouTube Creators	Aceite: Trailer explica o desafio de seis meses e aponta para o repositório. Evidência: youtube/trailer e brand-guide.md	3

Semana 02 - Arquitetura e ambiente profissional

Objetivo: Criar uma base técnica que suporte seis meses sem virar um projeto bagunçado.

Entrega: Aplicação inicia, CI passa e arquitetura está registrada.

RPG-M1-W02-01	Definir monólito modular Separe account, character, combat, inventory, progression, quest, world, economy e shared; use domínio, aplicação, infraestrutura e interfaces dentro de cada módulo. Architecture Decision Records	Aceite: O diagrama mostra módulos e nenhuma regra depende de controller ou JPA. Evidência: docs/architecture/modules.md	4
RPG-M1-W02-02	Criar projeto Java 21 e Spring Boot Gere Maven Wrapper com Web, Validation, Data JPA, Security, Actuator, PostgreSQL, Flyway e testes. Spring Boot System Requirements	Aceite: ./mvnw clean verify executa e /actuator/health responde UP. Evidência: pom.xml e src/	4
RPG-M1-W02-03	Criar árvore de pacotes Implemente o pacote io.github.arthurtvrs10.rpgarena e a estrutura modular definida, ainda sem lógica falsa. Java Learning	Aceite: Pacotes seguem o desenho e não existe pacote global service/controller/repository. Evidência: src/main/java/.../rpgarena	3
RPG-M1-W02-04	Registrar ADRs iniciais Crie ADRs para Java 21, Spring Boot 4.1, Maven, PostgreSQL, REST, monólito modular e UUID. Architecture Decision Records	Aceite: Cada ADR contém contexto, opções, decisão, consequências e status. Evidência: docs/adr/0001-0007	4
RPG-M1-W02-05	Configurar qualidade inicial Crie GitHub Actions com Java 21, cache Maven, verify, formatação e branch protection; valide Docker Desktop ou registre fallback com PostgreSQL local. GitHub Actions	Aceite: PR de teste falha com teste quebrado e passa após correção; ambiente possui plano A e B. Evidência: .github/workflows/ci.yml e docs/setup.md	5
RPG-M1-W02-06	Publicar episódio 1 Mostre escopo, decisões, árvore de módulos, setup e CI; use commits reais e feche com o próximo desafio. YouTube Creators	Aceite: Vídeo publicado, capítulos adicionados e links do repositório e playlist conferidos. Evidência: Episódio 1 publicado	4

Semana 03 - Domínio de personagem

Objetivo: Modelar regras usando objetos ricos, invariantes e casos de uso.

Entrega: Personagem consistente, independente de Spring e banco.

RPG-M1-W03-01	Criar Value Objects do personagem Implemente CharacterId, CharacterName, Health, Mana, Strength, Defense e Agility com validação no construtor/factory. Java Learning	Aceite: Valores inválidos não existem e igualdade segue valor, não referência. Evidência: character/domain/valueobject	5
RPG-M1-W03-02	Modelar classes e atributos iniciais Crie Warrior, Mage e Rogue como enum/política; defina stats iniciais e crescimento sem if espalhado. Java Learning	Aceite: Adicionar nova classe exige uma extensão localizada e testes deixam regras legíveis. Evidência: character/domain/archetype	4
RPG-M1-W03-03	Implementar agregado Character Controle nome, classe, nível, XP, vida atual e estados ACTIVE, DEFEATED e UNAVAILABLE dentro do agregado. Java Learning	Aceite: Métodos impedem cura acima do máximo, dano negativo e ações quando derrotado. Evidência: character/domain/model/Character.java	5
RPG-M1-W03-04	Criar factory, portas e casos de uso Implemente criar, obter e renomear personagem com input/output próprios e porta CharacterRepository. ArchUnit User Guide	Aceite: Casos de uso não importam web/JPA e retornam erro explícito. Evidência: character/application	5
RPG-M1-W03-05	Cobrir invariantes do personagem Use JUnit e testes parametrizados para nomes, atributos, classes, dano, cura, derrota e renomeação. JUnit User Guide	Aceite: Caminho feliz, erro e limite de cada regra crítica estão cobertos. Evidência: character/domain tests	4
RPG-M1-W03-06	Produzir Short 3 e capturar episódio 2 Grave Value Object vs tipo primitivo, factory e testes; salve antes/depois do domínio anêmico. YouTube Creators	Aceite: Short publicado e pasta do episódio contém gravações nomeadas. Evidência: youtube/ep-02/raw e Short 03	3

Semana 04 - Motor de combate determinístico

Objetivo: Entregar a primeira versão jogável com regras testáveis.

Entrega: Batalha completa e release v0.1.0.

RPG-M1-W04-01	Especificar regras de combate Defina iniciativa, turnos, ataque, defesa, crítico, habilidade, fuga, vitória, derrota e empate com exemplos numéricos. GitHub Documentation	Aceite: Fórmulas, arredondamentos e casos extremos não possuem ambiguidade. Evidência: docs/game-design/combat.md	4
RPG-M1-W04-02	Criar políticas de dano e aleatoriedade Separe DamagePolicy e RandomSource; injete seed/fake nos testes e proíba Math.random fora do adapter. Java Learning	Aceite: Mesma semente produz o mesmo histórico e dano nunca fica abaixo do mínimo. Evidência: combat/domain/policy	5
RPG-M1-W04-03	Modelar Battle e Turn Crie participantes, rodada, turno atual, estado, histórico e resultado como agregado. Java Learning	Aceite: Só o participante correto age e batalha encerrada é imutável. Evidência: combat/domain/model	6
RPG-M1-W04-04	Implementar ações e efeitos Implemente ataque, defesa, habilidade, item, fuga e efeitos poison/stun com duração controlada. Java Learning	Aceite: Cada ação valida custo/alvo e gera evento de batalha auditável. Evidência: combat/domain/action e effect	6
RPG-M1-W04-05	Fechar suíte e release v0.1 Teste vitória, derrota, crítico, dano mínimo, fuga, turno inválido, cooldown e status; atualize changelog e tag. JUnit User Guide	Aceite: Suite reproduz batalha completa e release v0.1.0 pode ser executada pelo README. Evidência: Tests, CHANGELOG e release v0.1.0	6
RPG-M1-W04-06	Publicar episódio 2 Conte a evolução do domínio anêmico para Character e Battle ricos; mostre testes determinísticos e demo final. YouTube Creators	Aceite: Vídeo publicado com capítulos, thumbnail e link para v0.1.0. Evidência: Episódio 2 publicado	4

Mês 2 - Persistência e progressão

Objetivo: Persistir dados e entregar inventário, equipamentos, XP, habilidades e missões.

Marco: v0.2.0 - jornada persistente

Prova: Banco nasce por migrations e a jornada sobrevive a reinício.

Semana 05 - PostgreSQL, Flyway e repositories

Objetivo: Persistir o núcleo sem contaminar o domínio.

Entrega: Banco criado por migrations e repositories testados.

RPG-M2-W05-01	Desenhar modelo relacional Mapeie account, character, stats, battle e battle_event; defina PK, FK, unique, checks e índices iniciais. PostgreSQL Documentation	Aceite: ERD corresponde aos agregados e cada constraint tem justificativa. Evidência: docs/database/erd-v1.drawio e PNG	5
RPG-M2-W05-02	Criar baseline Flyway Crie schema e migrations V1-V3 para personagens e batalhas; mantenha scripts imutáveis após aplicação. Flyway Documentation	Aceite: Banco vazio chega ao schema atual apenas ao iniciar a aplicação. Evidência: src/main/resources/db/migration	5
RPG-M2-W05-03	Mapear entidades JPA Crie modelos de persistência e mappers separados das entidades de domínio; configure UUID, version e timestamps. Spring Data JPA Reference	Aceite: O pacote domain não importa jakarta.persistence e round-trip mantém dados. Evidência: character/combat infrastructure persistence	6
RPG-M2-W05-04	Implementar adapters de repositório Implemente portas com Spring Data internamente; trate not found, unique e lock sem vazar DataAccessException. Spring Data JPA Reference	Aceite: Casos de uso trabalham somente com interfaces do domínio/aplicação. Evidência: Repository adapters	5
RPG-M2-W05-05	Testar com PostgreSQL real Use Testcontainers para migrations, save/load, constraints, versionamento e consultas de histórico. Testcontainers for Java	Aceite: Testes não usam H2 e executam localmente e no CI. Evidência: Persistence integration tests	5
RPG-M2-W05-06	Produzir Short 5 e capturar episódio 3 Mostre por que não expor JPA no domínio, migration real e banco nascendo do zero. YouTube Creators	Aceite: Short publicado e gravações de migration/repository separadas por cena. Evidência: youtube/ep-03/raw e Short 05	3

Semana 06 - Inventário, equipamento e transações

Objetivo: Garantir consistência ao manipular itens e atributos.

Entrega: Inventário transacional com concorrência testada.

RPG-M2-W06-01	Modelar catálogo de itens Crie ItemDefinition para arma, armadura, consumível e material com raridade, preço, efeito e restrições. Java Learning	Aceite: Tipos inválidos não são instanciados e catálogo é separado da posse. Evidência: inventory/domain/item	5
RPG-M2-W06-02	Implementar slots e pilhas Controle capacidade, stack size, adicionar, remover, transferir e consumir sem quantidade negativa. Java Learning	Aceite: Excesso de capacidade e item inexistente retornam erro de domínio sem alterar estado. Evidência: inventory/domain/inventory	6
RPG-M2-W06-03	Implementar equipar e desequipar Valide slot, classe e nível; calcule atributos derivados e preserve o item anterior se falhar. JUnit User Guide	Aceite: Equipamento muda apenas stats derivados e rollback lógico mantém consistência. Evidência: inventory/domain/equipment	5
RPG-M2-W06-04	Criar tabela de recompensas Modele peso, raridade e quantidade com RandomSource; valide pesos e permita seed nos testes. JUnit User Guide	Aceite: Distribuição é reproduzível e configuração inválida falha cedo. Evidência: inventory/domain/loot	4
RPG-M2-W06-05	Persistir e testar concorrência Faça loot, inventário e equipamento em transações; use optimistic locking e simule duas atualizações simultâneas. PostgreSQL Documentation	Aceite: Uma falha não deixa estado parcial e conflito vira resposta de negócio controlada. Evidência: Migrations e integration tests	7
RPG-M2-W06-06	Publicar episódio 3 Mostre PostgreSQL, Flyway, mapeamento, transação e teste concorrente do inventário. YouTube Creators	Aceite: Vídeo publicado com demo de rollback e link para PRs. Evidência: Episódio 3 publicado	4

Semana 07 - Progressão, níveis e habilidades

Objetivo: Criar evolução de personagem extensível e auditável.

Entrega: XP, level up e habilidades persistentes.

RPG-M2-W07-01	Definir curva de experiência Documente XP por nível, limite, excesso e atributos ganhos por classe; gere exemplos até o nível 20. Java Learning	Aceite: A curva é calculável, monotônica e possui tabela de referência. Evidência: docs/game-design/progression.md	4
RPG-M2-W07-02	Implementar ganho de XP Permita vários level ups em uma recompensa; atualize stats e recursos pela política da classe. Java Learning	Aceite: XP não se perde, limite máximo é respeitado e múltiplos níveis têm teste. Evidência: progression/domain	5
RPG-M2-W07-03	Modelar habilidades Crie SkillDefinition, custo, alvo, efeito, cooldown e pré-requisitos de classe/nível. Java Learning	Aceite: Uso inelegível não gasta recurso e explica a regra violada. Evidência: skill/domain	6
RPG-M2-W07-04	Publicar eventos de domínio Gere CharacterLeveledUp e SkillUnlocked; processe handlers após sucesso sem acoplar módulos. Spring Boot Reference	Aceite: Handlers podem falhar de forma observável sem corromper o agregado. Evidência: shared/domain/events e handlers	5
RPG-M2-W07-05	Persistir e testar progressão Crie migrations, adapters e testes parametrizados para curva, múltiplos níveis, skills, mana e cooldown. Testcontainers for Java	Aceite: Todos os limites documentados têm teste e persistência mantém estado. Evidência: Progression integration tests	6
RPG-M2-W07-06	Produzir Short 7 e capturar episódio 4 Grave XP de múltiplos níveis, política por classe e evento de domínio com teste. YouTube Creators	Aceite: Short publicado e roteiro visual do episódio revisado. Evidência: youtube/ep-04/raw e Short 07	3

Semana 08 - Missões e jornada persistente

Objetivo: Conectar combate, loot e progressão sem acoplamento direto.

Entrega: Missão completa e release v0.2.0.

RPG-M2-W08-01	Separar definição e instância Modele QuestDefinition imutável e QuestInstance por personagem com versão do conteúdo. Java Learning	Aceite: Alterar catálogo não muda progresso antigo silenciosamente. Evidência: quest/domain/model	5
RPG-M2-W08-02	Criar máquina de estados Implemente AVAILABLE, ACTIVE, COMPLETED, FAILED e CLAIMED com transições explícitas. JUnit User Guide	Aceite: Não é possível concluir sem objetivos nem resgatar duas vezes. Evidência: quest/domain/state	5
RPG-M2-W08-03	Atualizar progresso por eventos Consuma BattleWon, ItemCollected e LocationVisited; registre eventId para idempotência. Spring Boot Reference	Aceite: Evento repetido não duplica progresso e só missões elegíveis avançam. Evidência: quest/application/handlers	6
RPG-M2-W08-04	Entregar recompensa atomicamente Atualize XP, moedas e itens em uma transação; defina comportamento para inventário cheio. Spring Data JPA Reference	Aceite: Toda recompensa é aplicada uma vez ou nenhuma alteração persiste. Evidência: ClaimQuestRewardUseCase	6
RPG-M2-W08-05	Fechar jornada e v0.2.0 Recrie banco, rode migrations e execute personagem -> batalha -> loot -> XP -> missão; atualize docs, changelog e tag. Keep a Changelog	Aceite: Jornada sobrevive a reinício e um terceiro a reproduz pelo README. Evidência: Release v0.2.0	6
RPG-M2-W08-06	Publicar episódio 4 Explique progressão, habilidades, eventos e missão idempotente; mostre jornada persistente. YouTube Creators	Aceite: Vídeo, capítulos, thumbnail e release v0.2.0 estão publicados. Evidência: Episódio 4 publicado	4

Mês 3 - API e segurança

Objetivo: Expor o jogo por REST com autenticação, autorização e ownership.

Marco: v0.3.0 - API segura

Prova: Bruno executa cadastro, login, personagem, batalha e missão.

Semana 09 - Contrato REST, DTOs e Bruno

Objetivo: Criar uma API previsível antes de expor todos os recursos.

Entrega: Padrão HTTP, erros, validação e coleção Bruno.

RPG-M3-W09-01	Definir convenções REST Padronize recursos, verbos, status, paginação, ordenação, datas, UUID, versionamento e idempotency-key. MDN HTTP	Aceite: Documento possui exemplos de sucesso e erro para cada convenção. Evidência: docs/api/conventions.md	4
RPG-M3-W09-02	Criar DTOs e mapeadores Separe request/response do domínio e JPA; crie mappers testados e respostas sem dados internos. Spring Boot Reference	Aceite: Nenhum controller retorna entidade JPA ou senha/token indevido. Evidência: interfaces/rest/dto e mapper	5
RPG-M3-W09-03	Implementar validação em camadas Use Bean Validation para formato e domínio para regras; padronize mensagens e campos. Spring Framework Validation	Aceite: Payload inválido retorna 400 com lista de violações estável. Evidência: Validation annotations e tests	4
RPG-M3-W09-04	Criar Problem Details global Implemente code, title, detail, status, path, timestamp, correlationId e fieldErrors. Spring Boot Reference	Aceite: 400, 401, 403, 404, 409 e 500 seguem o mesmo formato. Evidência: GlobalExceptionHandler	5
RPG-M3-W09-05	Montar coleção e ambientes Crie local/public, baseUrl, token, pastas por recurso e asserts de status/body; versione com o código. Bruno Documentation	Aceite: Collection Runner executa health e primeiro CRUD sem editar URL. Evidência: bruno/RPG-Arena	4
RPG-M3-W09-06	Produzir Short 9 e capturar episódio 5 Mostre DTO vs entidade, erro padronizado e ambiente Bruno. YouTube Creators	Aceite: Short publicado e requests principais gravados em 1080p. Evidência: youtube/ep-05/raw e Short 09	3

Semana 10 - APIs de personagem e inventário

Objetivo: Expor os primeiros recursos com consultas e concorrência controladas.

Entrega: Personagem e inventário consumíveis por HTTP.

RPG-M3-W10-01	Criar endpoints de personagem Implemente POST, GET lista própria, GET por id e PATCH nome com DTOs e casos de uso. Spring Boot Reference	Aceite: Bruno cobre 201, 200, 400, 404 e 409. Evidência: /api/v1/characters	6
RPG-M3-W10-02	Criar endpoints de inventário Implemente listar, equipar, desequipar e consumir mantendo transações no application service. Spring Boot Reference	Aceite: Responses mostram atributos derivados e falhas não deixam estado parcial. Evidência: /api/v1/characters/{id}/inventory	6
RPG-M3-W10-03	Adicionar paginação e filtros Use limites máximos, ordenação allowlist e filtros por classe/status/raridade. Spring Data JPA Reference	Aceite: Parâmetros inválidos retornam 400 e consultas não usam full scan desnecessário. Evidência: Query services e indexes	5
RPG-M3-W10-04	Aplicar optimistic locking e idempotência Use @Version e chaves idempotentes em comandos com risco de repetição. PostgreSQL Documentation	Aceite: Requisições simultâneas não silenciam perda de atualização nem duplicam efeitos. Evidência: Concurrency controls	5
RPG-M3-W10-05	Integrar OpenAPI e testes HTTP Documente contratos e use testes com PostgreSQL para payloads, transações e versionamento. springdoc-openapi	Aceite: OpenAPI corresponde à implementação e CI executa testes HTTP. Evidência: OpenAPI inicial e integration tests	6
RPG-M3-W10-06	Publicar episódio 5 Mostre contrato REST, DTOs, Problem Details, Bruno e concorrência na API de personagem. YouTube Creators	Aceite: Vídeo publicado e coleção Bruno linkada no repositório. Evidência: Episódio 5 publicado	4

Semana 11 - Contas, JWT e sessões

Objetivo: Autenticar sem expor credenciais e preparar autorização real.

Entrega: Cadastro, login, refresh e logout seguros.

RPG-M3-W11-01	Modelar Account e roles Implemente email normalizado, passwordHash, status e roles PLAYER/ADMIN; adicione unique e timestamps. Spring Security Reference	Aceite: Email é único e conta bloqueada não autentica. Evidência: account domain e migration	5
RPG-M3-W11-02	Implementar cadastro Valide senha, use PasswordEncoder e respostas neutras; nunca registre senha. OWASP Cheat Sheet Series	Aceite: Banco só armazena hash e logs/responses não contêm segredo. Evidência: POST /api/v1/auth/register	5
RPG-M3-W11-03	Implementar login JWT Emita access token curto com sub, roles, issuer e audience; valide assinatura e expiração. RFC 7519 - JSON Web Token	Aceite: Tokens adulterados, expirados ou de issuer incorreto são rejeitados. Evidência: POST /api/v1/auth/login	6
RPG-M3-W11-04	Implementar refresh rotativo Persista hash do refresh, família, expiração e revogação; detecte reutilização. Spring Security Reference	Aceite: Refresh reutilizado revoga a família e logout encerra a sessão. Evidência: refresh_token table e endpoints	6
RPG-M3-W11-05	Criar matriz de permissões Defina público, autenticado, owner e ADMIN por endpoint e comando; configure deny by default. Spring Security Reference	Aceite: Matriz cobre todos os recursos e tem testes allow/deny. Evidência: docs/security/access-matrix.md	4
RPG-M3-W11-06	Produzir Short 11 e capturar episódio 6 Grave hash, access vs refresh e um token adulterado sendo recusado. YouTube Creators	Aceite: Short publicado e cenas de segurança não exibem segredo real. Evidência: youtube/ep-06/raw e Short 11	3

Semana 12 - Ownership e gameplay seguro

Objetivo: Impedir acesso horizontal e expor batalha/missões com segurança.

Entrega: API segura e release v0.3.0.

RPG-M3-W12-01	Aplicar ownership de personagem Valide conta autenticada no caso de uso, não apenas na URL/controller; admin usa regra explícita. Spring Security Reference	Aceite: Trocar characterId nunca concede acesso a outro jogador. Evidência: OwnershipPolicy	6
RPG-M3-W12-02	Expor batalha Implemente iniciar, agir e consultar com idempotência, optimistic locking e histórico paginado. MDN HTTP	Aceite: Ação fora de turno, duplicada ou de não proprietário é recusada. Evidência: /api/v1/battles	7
RPG-M3-W12-03	Expor missões Implemente listar, aceitar, acompanhar, abandonar e resgatar usando conta/personagem autenticados. Spring Boot Reference	Aceite: Recompensa não duplica e missão alheia retorna 403/404 conforme política. Evidência: /api/v1/quests	6
RPG-M3-W12-04	Criar suíte de segurança Teste 401, 403, ownership, roles, token expirado, refresh reutilizado e concorrência com PostgreSQL. Testcontainers for Java	Aceite: Todos os cenários allow/deny rodam no CI. Evidência: Security integration tests	7
RPG-M3-W12-05	Fechar v0.3.0 Rode Bruno do cadastro à missão, exporte OpenAPI, atualize README/changelog e publique tag. Keep a Changelog	Aceite: Pessoa externa joga por HTTP usando apenas documentação e coleção. Evidência: Release v0.3.0	5
RPG-M3-W12-06	Publicar episódio 6 Demonstre JWT, ownership, batalha por HTTP, testes de 403 e jornada Bruno. YouTube Creators	Aceite: Vídeo publicado com aviso para nunca versionar segredos. Evidência: Episódio 6 publicado	4

Mês 4 - Mundo e conteúdo

Objetivo: Adicionar exploração, encontros, NPCs, economia e missões ramificadas.

Marco: v0.4.0 - sistemas de mundo

Prova: Fluxo ponta a ponta inclui viagem, compra e escolha narrativa.

Semana 13 - Mundo, locais e exploração

Objetivo: Transformar módulos isolados em uma jornada de mundo.

Entrega: Mapa em grafo, viagem e encontros.

RPG-M4-W13-01	Modelar regiões e locais Crie Region, Location, nível recomendado, tags, disponibilidade e conexões direcionadas. Java Learning	Aceite: Grafo rejeita conexão inválida e localização inexistente. Evidência: world/domain/location	5
RPG-M4-W13-02	Implementar viagem Valide origem, destino conectado, custo, estado e requisito; publique LocationVisited. JUnit User Guide	Aceite: Não existe teletransporte indevido e custo é aplicado uma vez. Evidência: TravelUseCase	5
RPG-M4-W13-03	Criar encontros configuráveis Modele batalha, descoberta, NPC e recompensa com pesos e condições. Java Learning	Aceite: RandomSource torna encontro reproduzível e só opções elegíveis entram no sorteio. Evidência: world/domain/encounter	6
RPG-M4-W13-04	Persistir mapa e posição Crie migrations para local, aresta e posição; carregue seed versionado. Flyway Documentation	Aceite: Banco novo recebe mapa válido e posição sobrevive a reinício. Evidência: World migrations e seed	5
RPG-M4-W13-05	Expor mapa e viagem Implemente locais, destinos e travel; teste requisitos, ownership e eventos. Bruno Documentation	Aceite: Bruno demonstra exploração completa e bloqueios. Evidência: /api/v1/world e tests	6
RPG-M4-W13-06	Produzir Short 13 e capturar episódio 7 Mostre mapa como grafo, uma viagem negada e um encontro determinístico. YouTube Creators	Aceite: Short publicado e diagrama do mapa exportado. Evidência: youtube/ep-07/raw e Short 13	3

Semana 14 - NPCs, loja e economia

Objetivo: Criar transações econômicas sem exploit ou inconsistência.

Entrega: Compra e venda concorrentes e seguras.

RPG-M4-W14-01	Modelar Money e preços Use unidade inteira, moeda única, políticas de compra/venda, limites e arredondamento explícito. Java Learning	Aceite: Não há double, valor negativo, overflow silencioso ou preço fora da política. Evidência: economy/domain/Money	4
RPG-M4-W14-02	Modelar NPC e Merchant Separe NPC, diálogo, localização, catálogo, estoque e horário/condição de atendimento. Java Learning	Aceite: Comerciante só atende personagem elegível no mesmo local. Evidência: npc e economy domain	5
RPG-M4-W14-03	Implementar compra transacional Verifique saldo, estoque e inventário; debite e entregue item em uma transação. Spring Data JPA Reference	Aceite: Falha em qualquer etapa não consome moeda nem estoque. Evidência: PurchaseUseCase	6
RPG-M4-W14-04	Implementar venda e reposição Valide propriedade e item vendável; proteja último item com lock/versão e idempotência. PostgreSQL Documentation	Aceite: Venda repetida e estoque negativo são impossíveis sob concorrência. Evidência: Sell e Restock use cases	6
RPG-M4-W14-05	Testar economia concorrente Simule duas compras do último item, rollback e saldo insuficiente com Testcontainers. Testcontainers for Java	Aceite: Somente uma compra vence e todos os saldos fecham. Evidência: Economy integration tests	6
RPG-M4-W14-06	Publicar episódio 7 Mostre mundo, encontro, Money, compra atômica e teste do último item. YouTube Creators	Aceite: Vídeo publicado com diagrama da transação e demo. Evidência: Episódio 7 publicado	4

Semana 15 - Missões ramificadas e diálogo

Objetivo: Permitir conteúdo narrativo sem hardcode no fluxo principal.

Entrega: Escolhas, consequências e conteúdo versionado.

RPG-M4-W15-01	Definir schema narrativo Modele quest, etapas, escolhas, condições, diálogos, efeitos e recompensas em formato versionável. OpenAPI Specification	Aceite: Conteúdo inválido falha na carga com mensagem e caminho do campo. Evidência: docs/content/quest-schema.md	5
RPG-M4-W15-02	Implementar pré-requisitos e cadeias Crie policies para nível, classe, missão anterior, item, reputação e localização. Java Learning	Aceite: Sistema explica por que uma missão está bloqueada sem if espalhado. Evidência: QuestEligibilityPolicy	6
RPG-M4-W15-03	Modelar nós e escolhas Implemente DialogueNode, Choice e transições; valide opção disponível e consumo único. JUnit User Guide	Aceite: Escolha inexistente, bloqueada ou repetida não altera estado. Evidência: dialogue/domain	6
RPG-M4-W15-04	Aplicar efeitos e versionamento Permita moeda, item, reputação, flag e nova missão; vincule instância à versão publicada. Spring Data JPA Reference	Aceite: Nova publicação não corrompe jornada em andamento. Evidência: Narrative effects e content version	6
RPG-M4-W15-05	Expor narrativa e validar contratos Implemente dialogue/choices e testes de schema, cadeia, escolha e idempotência. Bruno Documentation	Aceite: Fluxo ramificado completo roda no Bruno e no CI. Evidência: /api/v1/dialogues e tests	6
RPG-M4-W15-06	Produzir Short 15 e capturar episódio 8 Mostre máquina de estados, uma escolha com consequência e conteúdo inválido sendo rejeitado. YouTube Creators	Aceite: Short publicado e árvore narrativa exportada. Evidência: youtube/ep-08/raw e Short 15	3

Semana 16 - Administração e publicação de conteúdo

Objetivo: Gerenciar conteúdo sem editar o banco manualmente.

Entrega: API administrativa e release v0.4.0.

RPG-M4-W16-01	Definir RBAC administrativo Mapeie permissões para itens, missões, locais, NPCs, publicação e auditoria; negue por padrão. Spring Security Reference	Aceite: PLAYER não acessa endpoints admin e testes cobrem cada permissão. Evidência: docs/security/admin-matrix.md	4
RPG-M4-W16-02	Criar CRUD de conteúdo Implemente criar, editar, consultar e desativar itens, missões, locais e comerciantes. Spring Boot Reference	Aceite: Operações usam DTO, validação, paginação e auditoria. Evidência: /api/v1/admin	7
RPG-M4-W16-03	Criar rascunho e publish Separe DRAFT, PUBLISHED e ARCHIVED; valide conteúdo inteiro antes de publicar. Spring Data JPA Reference	Aceite: Somente versão válida fica ativa e instâncias antigas mantêm referência. Evidência: ContentPublicationService	6
RPG-M4-W16-04	Registrar ações administrativas Grave actor, ação, recurso, versão, horário e correlationId sem segredos. Spring Boot Actuator	Aceite: É possível identificar quem alterou e publicou cada versão. Evidência: audit_event e logs	5
RPG-M4-W16-05	Executar ArchUnit e fechar v0.4 Impeça dependências proibidas, rode regressão, atualize docs/changelog e publique release. ArchUnit User Guide	Aceite: Build falha em violação arquitetural e v0.4.0 demonstra conteúdo administrável. Evidência: Architecture tests e release v0.4.0	6
RPG-M4-W16-06	Publicar episódio 8 Mostre missão ramificada, content version, RBAC e publicação administrativa. YouTube Creators	Aceite: Vídeo publicado com fluxo draft -> publish -> jogo. Evidência: Episódio 8 publicado	4

Mês 5 - Qualidade e operação

Objetivo: Elevar testes, Docker, documentação, observabilidade, desempenho e segurança.

Marco: v1.0.0-rc.1 - candidato a release

Prova: CI, Docker, OpenAPI, métricas e quality gates funcionam.

Semana 17 - Estratégia de testes e cobertura útil

Objetivo: Criar uma rede de segurança baseada em riscos, não em números vazios.

Entrega: Suíte organizada, rápida e auditável.

RPG-M5-W17-01	Escrever estratégia de testes Mapeie riscos por módulo e escolha unit, slice, integration, contract e smoke; defina o que não testar. JUnit User Guide	Aceite: Cada risco crítico possui tipo, camada, dado e objetivo de teste. Evidência: docs/testing/strategy.md	4
RPG-M5-W17-02	Completar testes de domínio Feche gaps de combate, inventário, progressão, quest, economia e policies; remova testes de getter. JUnit User Guide	Aceite: Cada regra crítica possui caminho feliz, erro e limite. Evidência: Domain test suites	7
RPG-M5-W17-03	Criar Test Data Builders Substitua fixtures globais por builders explícitos e factories por domínio. Mockito Documentation	Aceite: Testes ficam independentes, legíveis e sem estado compartilhado. Evidência: src/test/.../fixtures	5
RPG-M5-W17-04	Padronizar Testcontainers Crie base reutilizável, PostgreSQL isolado, Flyway real e limpeza determinística. Testcontainers for Java	Aceite: Suíte roda local/CI sem banco instalado e sem ordem entre testes. Evidência: Integration test infrastructure	6
RPG-M5-W17-05	Configurar JaCoCo Defina limites por pacote com exigência maior no domínio; publique relatório no CI. JaCoCo Documentation	Aceite: Queda abaixo do limite bloqueia PR e exclusões estão justificadas. Evidência: JaCoCo report e quality gate	5
RPG-M5-W17-06	Produzir Short 17 e capturar episódio 9 Compare teste útil vs cobertura vazia e mostre um builder reduzindo ruído. YouTube Creators	Aceite: Short publicado e relatórios/execuções gravados. Evidência: youtube/ep-09/raw e Short 17	3

Semana 18 - Docker e execução reproduzível

Objetivo: Fazer qualquer pessoa executar o projeto com um comando.

Entrega: Imagem, Compose, healthchecks e onboarding.

RPG-M5-W18-01	Criar Dockerfile multi-stage Compile com Maven, copie somente JAR, use JRE mínimo e usuário não root; crie .dockerignore. Docker Documentation	Aceite: Imagem inicia, não contém fonte/segredo desnecessário e executa sem root. Evidência: Dockerfile e .dockerignore	5
RPG-M5-W18-02	Criar Compose API + PostgreSQL Configure rede, volume, healthcheck, depends_on saudável e variáveis no .env.example. Docker Documentation	Aceite: docker compose up --build inicia ambiente do zero. Evidência: compose.yaml	5
RPG-M5-W18-03	Separar profiles e segredos Defina local, test e prod; valide variáveis obrigatórias e remova defaults inseguros. Spring Boot Reference	Aceite: Nenhum segredo real está no Git e produção falha cedo sem configuração. Evidência: application-*.yml	4
RPG-M5-W18-04	Adicionar readiness e liveness Configure Actuator, banco e estado da aplicação sem expor detalhes sensíveis. Spring Boot Actuator	Aceite: Compose e plataforma detectam indisponibilidade corretamente. Evidência: Health groups	4
RPG-M5-W18-05	Criar comandos e build de imagem Padronize build, test, run, logs, reset seguro e build Docker no CI. GitHub Actions	Aceite: Novo colaborador executa tudo em até 15 minutos e CI gera imagem. Evidência: scripts/, Makefile opcional e workflow	5
RPG-M5-W18-06	Publicar episódio 9 Mostre pirâmide de testes, PostgreSQL real, JaCoCo, Docker multi-stage e Compose. YouTube Creators	Aceite: Vídeo publicado e setup foi testado em clone limpo. Evidência: Episódio 9 publicado	4

Semana 19 - Documentação executável

Objetivo: Eliminar dependência de explicação verbal para usar ou contribuir.

Entrega: README, OpenAPI, Bruno e arquitetura atualizados.

RPG-M5-W19-01	Finalizar README principal Inclua proposta, demo, recursos, stack, arquitetura, setup, testes, API, roadmap, contribuição e licença. GitHub Documentation	Aceite: Pessoa externa entende e executa o projeto sem mensagem privada. Evidência: README.md	6
RPG-M5-W19-02	Completar OpenAPI Documente security schemes, endpoints, schemas, exemplos, paginação e Problem Details; valide no build. springdoc-openapi	Aceite: Todos os endpoints públicos aparecem e exemplos correspondem aos testes. Evidência: OpenAPI JSON/YAML e Swagger UI	6
RPG-M5-W19-03	Automatizar collection runner Crie fluxo health -> register -> login -> character -> battle -> quest; gere dados e asserts. Bruno Documentation	Aceite: Runner executa local e público sem edição manual. Evidência: bruno/RPG-Arena	5
RPG-M5-W19-04	Atualizar diagramas e ADRs Crie C4 contexto/container, módulos, ERD e sequência de batalha; marque ADRs substituídos. Mermaid Documentation	Aceite: Diagramas refletem o código e decisões possuem status correto. Evidência: docs/architecture e docs/adr	5
RPG-M5-W19-05	Criar guia de onboarding Explique ambiente, branches, commits, issue, testes, PR, DoD e debug comum. GitHub Documentation	Aceite: Contribuidor realiza primeira mudança seguindo apenas o guia. Evidência: CONTRIBUTING.md	4
RPG-M5-W19-06	Produzir Short 19 e capturar episódio 10 Mostre OpenAPI, Bruno Runner e diagrama que realmente acompanha o código. YouTube Creators	Aceite: Short publicado e assets de documentação organizados. Evidência: youtube/ep-10/raw e Short 19	3

Semana 20 - Observabilidade, desempenho e segurança

Objetivo: Transformar a aplicação em candidato a produção.

Entrega: Logs, métricas, revisão OWASP e release candidate.

RPG-M5-W20-01	Implementar logs estruturados Padronize level, correlationId, operation, duração e ids seguros; remova tokens/senhas. Spring Boot Actuator	Aceite: Uma requisição é rastreável do HTTP ao banco sem dado sensível. Evidência: Logging configuration	5
RPG-M5-W20-02	Adicionar métricas técnicas e de negócio Exponha latência, erros, pool, batalhas, vitórias e falhas com tags de baixa cardinalidade. Micrometer Reference	Aceite: Métricas respondem e não criam cardinalidade por usuário/personagem. Evidência: Micrometer metrics	5
RPG-M5-W20-03	Analisar queries e índices Detecte N+1, use EXPLAIN ANALYZE em consultas críticas e ajuste índices com evidência. PostgreSQL Documentation	Aceite: Relatório compara plano antes/depois e remove N+1. Evidência: docs/performance/database.md	6
RPG-M5-W20-04	Executar revisão OWASP Revise auth, autorização, validação, CORS, headers, rate limit, logs, segredos e dependências. OWASP Cheat Sheet Series	Aceite: Checklist possui evidência e issue para cada risco restante. Evidência: docs/security/review.md	6
RPG-M5-W20-05	Criar quality gates e RC Execute testes, cobertura, ArchUnit, build Docker e scan de dependências; publique v1.0.0-rc.1. GitHub Actions	Aceite: Main só recebe merge verde e RC passa regressão completa. Evidência: Workflow e release candidate	6
RPG-M5-W20-06	Publicar episódio 10 Mostre logs, métricas, N+1, EXPLAIN, checklist OWASP e quality gates. YouTube Creators	Aceite: Vídeo publicado com resultados antes/depois e limites honestos. Evidência: Episódio 10 publicado	4

Mês 6 - Publicação e portfólio

Objetivo: Publicar, estabilizar, abrir colaboração e transformar o projeto em prova profissional.

Marco: v1.0.0 - release pública

Prova: Projeto online, documentado, demonstrável e usado em candidaturas.

Semana 21 - Deploy e operação

Objetivo: Publicar com procedimentos de recuperação e verificação.

Entrega: API HTTPS, banco gerenciado, smoke e monitoramento.

RPG-M6-W21-01	Escolher plataforma e registrar ADR Compare custo, PostgreSQL, HTTPS, logs, métricas, secrets, rollback e limites; escolha uma opção. Architecture Decision Records	Aceite: ADR explica critérios, escolha, riscos e plano de saída. Evidência: docs/adr/deployment.md	4
RPG-M6-W21-02	Provisionar banco e aplicação Crie banco, usuário de menor privilégio, variáveis e deploy da imagem; não copie segredo para arquivo. Spring Boot Reference	Aceite: API responde por HTTPS e banco aceita apenas conexão necessária. Evidência: Ambiente público	7
RPG-M6-W21-03	Controlar migrations em produção Defina etapa única de migração, timeout, backup e compatibilidade entre versão nova/antiga. Flyway Documentation	Aceite: Deploy concorrente não executa migration perigosa duas vezes. Evidência: Pipeline/runbook de migration	5
RPG-M6-W21-04	Criar backup e rollback Documente backup, restore, rollback da imagem e resposta a migration incompatível; ensaie uma recuperação. PostgreSQL Documentation	Aceite: Runbook registra RPO/RTO aproximados e evidência do ensaio. Evidência: docs/operations/runbook.md	6
RPG-M6-W21-05	Automatizar smoke e alertas Teste health, register, login, character e read; alerte indisponibilidade, erros e banco. GitHub Actions	Aceite: Smoke pós-deploy passa e cada alerta aponta para ação do runbook. Evidência: Smoke workflow e alerts	6
RPG-M6-W21-06	Produzir Short 21 e capturar episódio 11 Mostre deploy, secrets, healthcheck, smoke falhando e rollback. YouTube Creators	Aceite: Short publicado sem expor credencial e gravações organizadas. Evidência: youtube/ep-11/raw e Short 21	3

Semana 22 - Governança open source

Objetivo: Permitir colaboração real com histórico profissional.

Entrega: Community files, templates e contribuição simulada.

RPG-M6-W22-01	Revisar licença e terceiros Confirme MIT, dependências e origem de assets; registre atribuições quando necessário. Choose a License	Aceite: Nenhum asset/dependência está sem licença compatível conhecida. Evidência: LICENSE e THIRD_PARTY_NOTICES.md	4
RPG-M6-W22-02	Adicionar conduta, segurança e suporte Crie CODE_OF_CONDUCT, SECURITY e SUPPORT com escopo, canais e resposta. Contributor Covenant	Aceite: Repositório informa comportamento, vulnerabilidades e dúvidas. Evidência: Community files	4
RPG-M6-W22-03	Criar templates de issue e PR Adicione bug, feature, task e PR com reprodução, aceite, teste e vínculo à issue. GitHub Documentation	Aceite: Toda nova issue/PR nasce com informações acionáveis. Evidência: .github templates	4
RPG-M6-W22-04	Organizar labels, milestones e board Use type, area, priority, difficulty e status; organize v1.0 e v1.1. GitHub Documentation	Aceite: Backlog público é filtrável e cada issue tem próximo passo. Evidência: GitHub Project	4
RPG-M6-W22-05	Simular contribuição externa Abra issue, branch, mudança pequena, PR, review, correção e squash merge. GitHub Documentation	Aceite: Histórico prova o fluxo profissional do início ao fim. Evidência: Issue e PR de simulação	5
RPG-M6-W22-06	Publicar episódio 11 Mostre deploy, runbook, templates e contribuição simulada sem transformar em tutorial de clique. YouTube Creators	Aceite: Vídeo publicado e repositório recebe chamadas para contribuição. Evidência: Episódio 11 publicado	4

Semana 23 - Estabilização e release candidate

Objetivo: Eliminar bloqueadores antes da versão 1.0.

Entrega: RC aprovada, changelog e relatório de carga.

RPG-M6-W23-01	Executar bug bash Teste jornadas, concorrência, payload hostil, indisponibilidade e permissões; abra issues com severidade. Bruno Documentation	Aceite: Cada falha possui passos, evidência, impacto e decisão. Evidência: Milestone v1.0 bug bash	6
RPG-M6-W23-02	Resolver blockers e regressões Trabalhe em PRs pequenos; toda correção crítica recebe teste de regressão. GitHub Documentation	Aceite: Nenhum blocker/critical fica aberto e CI permanece verde. Evidência: PRs de estabilização	8
RPG-M6-W23-03	Executar teste de carga mínimo Defina cenário, volume, ambiente e percentis; teste leitura e comando sem derrubar dados reais. Spring Boot Actuator	Aceite: Relatório contém p50/p95/p99, erros, gargalo e limite conhecido. Evidência: docs/performance/load-test.md	6
RPG-M6-W23-04	Fazer revisão final Revalide secrets, permissões, dependências, rate limit, logs e endpoints administrativos. OWASP Cheat Sheet Series	Aceite: Nenhum finding crítico/alto permanece sem mitigação documentada. Evidência: docs/security/final-review.md	5
RPG-M6-W23-05	Preparar changelog e upgrade Organize Added, Changed, Fixed, Security, breaking changes e instruções de migração. Keep a Changelog	Aceite: Changelog reconcilia PRs e um usuário da v0.4 atualiza para RC. Evidência: CHANGELOG.md e v1.0 checklist	5
RPG-M6-W23-06	Produzir Short 23 e capturar episódio final Grave bug real, teste de regressão, carga e checklist de release. YouTube Creators	Aceite: Short publicado e demo final possui backup de gravação. Evidência: youtube/ep-12/raw e Short 23	3

Semana 24 - Release 1.0, portfólio e mercado

Objetivo: Transformar o projeto concluído em prova pública de competência.

Entrega: v1.0.0, case study, demo e candidaturas iniciadas.

RPG-M6-W24-01	Publicar v1.0.0 Crie tag, release notes, imagem, deploy e smoke final; congele o marco e abra v1.1. Semantic Versioning	Aceite: Release pública corresponde ao commit implantado e smoke passa. Evidência: GitHub Release v1.0.0	6
RPG-M6-W24-02	Escrever estudo de caso Explique problema, escopo, arquitetura, regras difíceis, decisões, testes, incidentes, resultados e próximos passos. Architecture Decision Records	Aceite: Case possui evidências, links e decisões explicadas com suas palavras. Evidência: docs/portfolio/case-study.md	6
RPG-M6-W24-03	Gravar demonstração técnica Mostre setup, autenticação, personagem, combate, missão, testes, Docker e produção em 6-10 minutos. YouTube Creators	Aceite: Demo prova funcionamento sem depender de cortes confusos. Evidência: Vídeo demo no README	6
RPG-M6-W24-04	Atualizar GitHub e currículo Fixe o repositório, adicione case/demo, escreva bullets de impacto e liste tecnologias realmente usadas. GitHub Documentation	Aceite: Recrutador encontra contexto, código, execução e resultados em poucos minutos. Evidência: GitHub, currículo e LinkedIn	5
RPG-M6-W24-05	Iniciar candidaturas e feedback Candidate-se, registre vaga, resposta, gaps e ação; pratique explicar arquitetura, SQL, segurança e testes. GitHub Documentation	Aceite: Candidaturas começaram antes de qualquer sensação de perfeição e feedback vira issue/estudo. Evidência: Tracker de candidaturas e interview-notes.md	5
RPG-M6-W24-06	Publicar episódio 12 e encerrar a temporada Conte a história completa, resultados, decisões, erros, métricas, lições e roadmap v1.1. YouTube Creators	Aceite: Final publicado com playlist completa, CTA para repositório e retrospectiva após 7 dias. Evidência: Episódio 12 e playlist final	5

Minissérie YouTube

Um episódio principal a cada duas semanas e um Short por semana. Cada episódio apresenta problema, decisão, implementação, teste, resultado e aprendizado.

Ep.	Título e hook	Demo e foco	Thumbnail
01	Construindo um RPG backend profissional: escopo e arquitetura O maior erro não é escolher a tecnologia errada; é começar a codar sem saber o que precisa terminar.	Demo: Repositório, diagrama modular, aplicação iniciando e check verde. Foco: MVP e não objetivos, Monólito modular, Spring Boot + Java 21, ADRs e CI	RPG BACKEND DO ZERO
02	Personagem e combate: domínio rico em Java Se toda regra está no service, suas entidades são apenas tabelas com getters.	Demo: Batalha completa executada por teste com seed fixa. Foco: Value Objects, Agregados, Políticas, Testes determinísticos	ADEUS, DOMÍNIO ANÊMICO
03	PostgreSQL, Flyway e inventário transacional Um item duplicado por concorrência destrói a confiança no jogo.	Demo: Migration do zero, rollback e duas atualizações concorrentes. Foco: ERD, Flyway, Adapters JPA, Transações e locks	SEM DUPLICAR ITENS
04	XP, habilidades e missões com eventos de domínio Conectar tudo com chamadas diretas funciona até a primeira mudança.	Demo: Batalha gera XP, desbloqueia habilidade e conclui missão. Foco: Curva de XP, Skills, Eventos, Máquina de estados	O RPG COMEÇOU A EVOLUIR
05	Transformando o RPG em uma API REST previsível Uma API que funciona só no seu Postman não é uma API pronta.	Demo: CRUD de personagem e inventário com conflito de versão. Foco: REST, DTOs, Problem Details, Bruno e OpenAPI	API QUE OUTROS CONSEGUEM USAR
06	JWT, ownership e batalha segura por HTTP Ter JWT não impede um jogador de tentar acessar o personagem do outro.	Demo: Ataque com owner funciona; troca de ID retorna acesso negado. Foco: Password hashing, Access/refresh, Ownership, Security tests	JWT NÃO É AUTORIZAÇÃO
07	Mundo, viagens, NPCs e economia consistente A economia quebra quando duas pessoas compram o último item.	Demo: Viagem gera encontro e compra concorrente permite só um vencedor. Foco: Grafo de locais, Encontros, Money, Compra atômica	QUEM COMPRA O ÚLTIMO ITEM?
08	Missões ramificadas e conteúdo administrável Hardcode de diálogo transforma cada nova missão em deploy.	Demo: Admin publica missão; jogador escolhe e recebe consequência. Foco: Schema narrativo, Choices, Versionamento, RBAC admin	MISSÕES SEM HARDCODE
09	Testes reais e Docker: tornando o projeto reproduzível Cobertura alta não salva teste que nunca encontra um bug.	Demo: Clone limpo, Testcontainers, JaCoCo e Docker Compose. Foco: Estratégia de testes, Builders, Testcontainers, Docker	FUNCIONA EM OUTRA MÁQUINA?
10	Documentação, logs, métricas, performance e segurança Se você não consegue observar a falha, você não consegue operar o sistema.	Demo: CorrelationId, métrica, N+1 corrigido e quality gate. Foco: OpenAPI, Observabilidade, EXPLAIN, OWASP	DE API FUNCIONAL A OPERÁVEL
11	Deploy e open source com fluxo profissional Publicar não é apertar deploy; é saber voltar quando der errado.	Demo: Deploy, smoke, rollback e PR externo simulado. Foco: Deploy, Migrations, Runbook, Community files	COLOQUEI O RPG ONLINE
12	Release 1.0: seis meses construindo um RPG backend O resultado não é uma lista de tecnologias; é um sistema que eu consigo explicar e entregar.	Demo: Jornada completa em produção, testes, GitHub e release. Foco: Bug bash, Carga, Release, Portfólio e carreira	6 MESES. 1 RPG. TUDO O QUE APRENDI

Roteiro padrão de episódio

- 0:00-0:20 - consequência ou problema concreto.
- 0:20-0:50 - promessa do que será construído e comprovado.
- 0:50-2:00 - estado anterior e regra do produto.
- 2:00-5:00 - decisão, alternativa descartada e trade-off.
- 5:00-11:00 - construção com apenas os trechos essenciais.
- 11:00-14:00 - teste e demonstração executável.
- 14:00-16:00 - erro real, aprendizado e próxima etapa.
- Final - CTA para release, issue, documentação ou próximo episódio.

Checklist de publicação

- [ ] Sem segredo, token, e-mail privado ou notificação na tela.
- [ ] Thumbnail legível em tamanho pequeno e com no máximo quatro palavras.
- [ ] Capítulos, descrição, links, playlist e comentário fixado.
- [ ] Métricas registradas em 48 horas e sete dias.
- [ ] Código/release do vídeo reproduzível no repositório.

Critérios finais de sucesso

- [ ] Constrói e executa um backend completo com Java 21 e Spring Boot.
- [ ] Modela regras de personagem, combate, inventário, progressão, missão, mundo e economia com orientação a objetos.
- [ ] Usa PostgreSQL, SQL, migrations, índices, transações e concorrência com evidência.
- [ ] Entrega API REST com JWT, refresh, roles, ownership, validação e erros padronizados.
- [ ] Mantém testes unitários, integração com PostgreSQL, segurança, arquitetura, smoke e carga mínima.
- [ ] Valida jornadas no Bruno e publica OpenAPI utilizável.
- [ ] Trabalha com issues, branches, commits, PRs, reviews, CI, releases e changelog.
- [ ] Executa localmente por Docker Compose e publica ambiente HTTPS com runbook e rollback.
- [ ] Mantém README, ADRs, diagramas, guia de contribuição e documentação operacional.
- [ ] Explica decisões, trade-offs, erros e melhorias usando o próprio código.
- [ ] Publica 12 episódios, 24 Shorts e uma demonstração técnica final.
- [ ] Usa o projeto em currículo, GitHub, entrevistas e candidaturas sem esperar perfeição.

Regra de evidência

Cada critério final deve apontar para um teste, PR, release, diagrama, endpoint, vídeo ou ambiente publicado. Conhecimento sem evidência não conta como concluído.

Referências oficiais

java	Java Learning	https://dev.java/learn/
spring	Spring Boot Reference	https://docs.spring.io/spring-boot/
springRequirements	Spring Boot System Requirements	https://docs.spring.io/spring-boot/system-requirements.html
springData	Spring Data JPA Reference	https://docs.spring.io/spring-data/jpa/reference/
springSecurity	Spring Security Reference	https://docs.spring.io/spring-security/reference/
springValidation	Spring Framework Validation	https://docs.spring.io/spring-framework/reference/core/validation/beanvalidation.html
actuator	Spring Boot Actuator	https://docs.spring.io/spring-boot/reference/actuator/
maven	Apache Maven Guides	https://maven.apache.org/guides/
postgres	PostgreSQL Documentation	https://www.postgresql.org/docs/current/
flyway	Flyway Documentation	https://documentation.red-gate.com/fd
junit	JUnit User Guide	https://junit.org/junit5/docs/current/user-guide/
mockito	Mockito Documentation	https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html
testcontainers	Testcontainers for Java	https://java.testcontainers.org/
archunit	ArchUnit User Guide	https://www.archunit.org/userguide/html/000_Index.html
jacoco	JaCoCo Documentation	https://www.jacoco.org/jacoco/trunk/doc/
docker	Docker Documentation	https://docs.docker.com/
bruno	Bruno Documentation	https://docs.usebruno.com/
openapi	OpenAPI Specification	https://spec.openapis.org/oas/latest.html
springdoc	springdoc-openapi	https://springdoc.org/
http	MDN HTTP	https://developer.mozilla.org/en-US/docs/Web/HTTP
jwt	RFC 7519 - JSON Web Token	https://www.rfc-editor.org/rfc/rfc7519
owasp	OWASP Cheat Sheet Series	https://cheatsheetseries.owasp.org/
github	GitHub Documentation	https://docs.github.com/
actions	GitHub Actions	https://docs.github.com/actions
commits	Conventional Commits	https://www.conventionalcommits.org/en/v1.0.0/
semver	Semantic Versioning	https://semver.org/
changelog	Keep a Changelog	https://keepachangelog.com/en/1.1.0/
adr	Architecture Decision Records	https://adr.github.io/
mermaid	Mermaid Documentation	https://mermaid.js.org/
micrometer	Micrometer Reference	https://docs.micrometer.io/micrometer/reference/
license	Choose a License	https://choosealicense.com/
covenant	Contributor Covenant	https://www.contributor-covenant.org/version/2/1/code_of_conduct/
youtube	YouTube Creators	https://www.youtube.com/creators/

Próxima ação

Abra o arquivo da Semana 01, crie a primeira issue e execute a tarefa RPG-M1-W01-01. Não adicione escopo antes da release v1.0.0.